



4. Manejo de tiempos: Timers

Física e Inteligencia Artificial para Videojuegos

Doble Grado en Física e Inteligencia Artificial

Curso 2025 – 2026

Fernando Madrid Recio

Timer con Time.deltaTime.

- La forma más sencilla de contar tiempo en nuestro juego es a partir de **Time.deltaTime**.
- Recordemos:
 - **Time.deltaTime** = “Por segundo”.
- Por lo tanto:
 - Hacer un contador que cuente **1 por segundo**:

```
float timer = 0;

// Update is called once per frame
Mensaje de Unity | 0 referencias
void Update()
{
    //Temporizador que cuenta 1 por segundo.
    timer += 1 * Time.deltaTime;
}
```

Timer con Time.deltaTime.

- Se pueden aplicar este tipo de contadores en muchos contextos.
- Por ejemplo aplicar efectos cada cierto tiempo o incrementar variables en función del tiempo:

```
float timer = 0;
float vida = 200;
// Update is called once per frame
Mensaje de Unity | 0 referencias
void Update()
{
    timer += 1 * Time.deltaTime;
    //Si pasan 5 segundos...
    if(timer >= 5f)
    {
        //Me hacen daño...
        vida -= 10;
        //Y reseteo timer.
        timer = 0;
    }
}
```

```
float score = 0;
// Update is called once per frame
Mensaje de Unity | 0 referencias
void Update()
{
    //La puntuación incrementa 100 por segundo.
    score += 100 * Time.deltaTime;
}
```

A tener en cuenta

- Para que el timer pueda ser efectivo, ha de ser actualizado **siempre** en el Update().
- Si, por ejemplo, se desea inicializar un timer en el OnTriggerEnter, podemos enlazar dicho evento con un **bool**.
- Dicho bool controlará el timer.
- Los timers también pueden realizar cuentas **regresivas**.

```
bool poderActivo;
float tiempoPoder = 3;

// Update is called once per frame
Mensaje de Unity | 0 referencias
void Update()
{
    if(poderActivo)
    {
        tiempoPoder -= 1 * Time.deltaTime;
        if(tiempoPoder < 0)
        {
            tiempoPoder = 3;
            poderActivo = false;
        }
    }
}

Mensaje de Unity | 0 referencias
private void OnTriggerEnter(Collider other)
{
    if(other.gameObject.CompareTag("PowerUp"))
    {
        poderActivo = true;
    }
}
```

Modificando la vista de la cámara

- En un juego 2D, contamos por defecto con una cámara **ortográfica**, esto es, **sin profundidad** ni perspectiva.
- Si queremos modificar el color que aparece de fondo, editar el campo **Background**.
- Si queremos modificar cuánta área es capaz de ver nuestra cámara, editar el campo **Size**

